



WEEK 2 • LECTURE 2

Spec-Driven Development

Scott Davis, Muratcan Cicek, Sedanur Ceylan, Dr. Muhammed Davud

Today

01

The AI-native SDLC

02

Why do we need SDD?

03

**When SDD is not the
right fit**

04

SDD and human-in-the-loop

05

Plan Mode: the bridge

06

Three levels of maturity

07

What good specs look like

08

The SDD workflow

09

SDD frameworks

01 The AI-native SDLC

Code is no longer the bottleneck

LEFT AND RIGHT

The bottleneck moves

Planning, review and deployment now set the pace, not writing code.

CONTROLS

Human-speed controls stall

Approval steps built for weekly releases can't keep up with hourly changes.

GOVERNANCE

Governance costs spike

Every exception that goes to a committee becomes a queue.

Six stages, one artifact chain

`intent.md`

1 Plan

Agent turns an idea into `intent.md`.
The originator approves it.

`spec.md`

2 Design

Agent writes `spec.md` with org
rules applied. Product owner
signs off.

`plan.md` → `code`

3 Build

Agent writes `plan.md` before code.
The engineer interrogates the plan.

PASSING CHECKS

4 Test

Agent iterates against checks it can
run itself. Tests stay locked.

REVIEWED PR

5 Deploy

Agent pre-reviews the PR. Humans
judge intent and risk, and approve.

`new intent.md` ↺

6 Maintain

Alerts trigger diagnosis. A fix starts
as a new `intent.md`.

Put team knowledge where the agent can read it

ALWAYS LOADED

Project context file

Conventions, build and test commands, recurring mistakes. Under a page. Update it when the agent repeats an error.

LOADED WHEN RELEVANT

Skills

Team policies like security, API and brand rules. Versioned, owned by the policy owner. Advisory.

ENFORCED

Hooks

Scripts that block, allow or log an action. Use them for rules that must hold with no exceptions.

Give the agent a feedback loop

DEFINE DONE

One measurable target

“All tests pass.” “The screenshot matches the mock.”

LOCK THE TEST

Test first for bugs

Write the failing test, then block the agent from editing it.

EVALS

Test the setup too

Rerun evals whenever rules files or skills change.

METRICS

Measure the loop

First-pass CI rate, rework per change, change failure rate.

Autonomy goes down as risk goes up

Where	What the agent may do
Development	Act freely in a sandbox with scoped credentials.
Pull request	Review against policy and fix comments. A code owner approves.
Production	Nothing without a named human's approval. It never approves its own work.
Incidents	Small deviation: log. Larger: diagnose read-only. Severe: propose a fix as a PR.

02 Why do we need SDD?

“Fix the login bug.”

HUMAN

A developer pushes back

Which login flow? What does the user see? Can you reproduce it? Which environment?

AGENT

An agent starts coding

It picks the most typical “login bug” from its training data and fixes that. It solves the problem it imagined.

Why ambiguity costs more with agents

VELOCITY

50 → 500 lines

While a developer writes 50 lines on a wrong assumption, an agent writes 500, all built on the same guess.

SILENCE

No questions

A human stops and asks. An agent resolves the ambiguity on its own and keeps going.

Garbage in, clean-looking garbage out

It compiles and runs

The error is in the meaning, not the syntax. Shallow tests still pass.

Research agrees

Song et al. (2023): misreading the prompt is a top error type. Chen et al. (2024): seven kinds of spec misunderstanding.

Errors compound

One early wrong guess, like sync instead of async, ends up buried under 500 reasonable-looking lines.

You already write specs

PRD

What to build, for whom, and what success looks like

ADR

A decision, the alternatives considered, and why

RFC

A proposed change, open to critique before it's built

User story

Who needs what, and why, plus acceptance criteria

“Add authentication to the admin panel.”

- Which mechanism: OAuth2, JWT, sessions or magic link?
- How long do tokens last? Do we need refresh tokens?
- A token expires mid-session: redirect, or show an error?
- Unauthenticated requests: 401, 403, or both?
- What happens to sessions that exist today?
- Is there “stay logged in”? For how long?

Agents consume specs,
not user stories.

03 When SDD is not the right fit

Can't write acceptance criteria? You're still exploring.

STEP 1

Explore

Spike, prototype or a Plan Mode chat. The goal is learning.



STEP 2

Specify

What you learned becomes constraints, edge cases and criteria.



STEP 3

Build

Production code from the spec, on a fresh branch.

Why engineers skip the spec

Objection	Answer
Blank page	Start from four headings, not an empty document.
“Coding is faster”	Slower to start, faster to finish. A 20-minute spec can save a 2-hour fix-up loop.
Bias toward action	The cure for bad process is right-sized process.
Documentation hangover	These specs are short, and an agent reads them right away.
Maintenance	Update the spec in the same PR as the code: 5 to 10 minutes.

04 SDD and human-in-the-loop

Four human checkpoints

BEFORE PLANNING

Spec review

Highest leverage: minutes to fix here, hours once it's code.

BEFORE EXECUTION

Plan review

Skip it and a wrong architecture gets built, confidently.

DURING EXECUTION

Diff review

Does the behavior meet the criteria and fit the codebase?

BEFORE PRODUCTION

Deployment review

Not every production change can be undone.

Tell the agent when to stop and ask

Trigger	Stop and ask when...
Ambiguity	There are two valid approaches and it can't tell which fits the architecture.
Divergence	A test still fails after three attempts to fix it.
Contradiction	Existing behavior contradicts the acceptance criteria.
Irreversibility	It's about to delete or migrate data outside a test environment.

Let the agent interview you first

I want to build [feature].

Before writing anything, ask me as many questions as you need for a complete spec.md: goal, constraints, acceptance criteria, non-goals and edge cases.

Don't write the spec until I say so.

Answers replace guesses

Without the interview, the agent fills silences with what most teams do.

Make it a team activity

PM, designer and engineer answer the questions together.

05 Plan Mode: the bridge

Plan Mode: plan first, then build

```
> Add pagination to GET /api/orders.  
The frontend expects cursors.
```

PLAN

1. routes/orders.ts: accept cursor, limit
2. orderService.ts: WHERE on cursor, not OFFSET
3. Response adds nextCursor, hasMore
4. Update types in types/api.ts
5. Tests in tests/routes/orders.test.ts

```
? Cursor on order ID or createdAt?  
Assuming ID.
```

Read-only until you approve

Claude Code (Shift+Tab), Cursor and other agents.

Review it like a PR

What's missing? Here: the API docs. Its question is a gap in your spec.

Plan Mode is a bridge, not SDD

It's transient

The plan lives in one chat. A teammate can't pick it up tomorrow.

It answers how, not what

It takes your description at face value and won't surface missing requirements.

1

Plan Mode on every non-trivial task



2

Notice what you have to correct



3

Write those things in spec.md first



4

Use Plan Mode to check it read the spec

06 Three levels of maturity

Three levels of SDD maturity

LEVEL 1

Spec-first

Write the spec before the agent starts: goal, constraints, acceptance criteria.

Use for any task over 30 minutes of agent work.

LEVEL 2

Spec-anchored

The spec lives in the repo and changes in the same PR, updated first.

Use for code that keeps changing.

LEVEL 3

Spec-as-source

People edit only the spec; code is generated from it. Still theoretical at scale.

Watch it, don't adopt it yet.

Two tiers of specs

STRATEGIC

Rules file · ADRs · system requirements

Living documents. Updated when the system's direction changes; every agent session reads them.

TACTICAL

spec.md · plan.md · tasks.md per change

Drive one feature or fix, then stay as history. A follow-on feature gets a new spec.

07 What good specs look like

Start every spec from four headings

GOAL

What does success look like?

CONSTRAINTS

What must the implementation respect?

ACCEPTANCE CRITERIA

How will you verify it's done?

OUT OF SCOPE

What should the agent not touch?

Same request, two specs

BAD

“Add a caching layer to improve performance.”

The agent chooses the library, TTL, cache key, invalidation and monitoring for you.

Goal

Cache /api/products to cut DB load

Constraints

- Existing node-cache; no new infra
- TTL 5 min · key = category ID
- Invalidate on product.updated

Acceptance criteria

- 1st request hits the DB, then cache
- Invalidated within 1 second
- Hit/miss ratio on dashboards

Out of scope

- Distributed cache, other endpoints

Production-grade acceptance criteria

WHEN A STEP FAILS

Failure mode

If the save fails, the UI restores the last saved state.

WHEN TWO THINGS HAPPEN AT ONCE

Concurrency

Changing a preference doesn't affect emails already queued.

HOW YOU MEASURE IT

Observability

The skip rate is exported as a metric and watched after rollout.

HOW YOU UNDO IT

Rollback

Turning the flag off restores the old path within one minute.

Write criteria an agent can test

EARS • USED BY KIRO

WHEN a user fails to log in
5 times in 15 minutes
THE SYSTEM SHALL lock the
account for 15 minutes

GIVEN / WHEN / THEN

GIVEN a locked account
WHEN the user asks to reset
THEN a signed link is sent
AND it expires in 15 minutes

“Add a search bar to the products page.” Write the requirements.

FUNCTIONAL • 3

What the system does

What can the user search, and what do they get back?

NON-FUNCTIONAL • 3

How well it does it

How fast, how many users, which devices?
Give numbers.

08 The SDD workflow

One folder per change, three core files

```
specs/  
  001-notification-prefs/  
    spec.md  
    plan.md  
    tasks.md  
    risks.md  
  002-payment-retry/  
    ...
```

spec.md

What and why. No file paths or tech choices.

plan.md

How: approach, trade-offs, files, order.

tasks.md

Small tasks, each with a done check.

One short prompt per phase

SPECIFY

Ask me questions until you can write spec.md. Don't write it until I say so.

PLAN

Read spec.md and the code. Write plan.md: approach, trade-offs, files, order. No code.

TASKS

From plan.md write tasks.md: one change per task, a done check each.

EXECUTE

Implement tasks.md in order; tick each done check. Don't edit spec.md. Blocked? Stop.

Every task needs a done check

- [] PATCH /api/users/me/notifications
Done when: integration test for valid update, invalid keys and schema validation passes.
- [] NotificationService.send()
checks preferences.
Done when: email tests pass and a disabled type is not sent.

No done check, no finish line

The agent stops too early, or never stops.

Split megatasks

If the done check says “and” more than once, split the task.

Existing code: start with spec islands

1

Spec what you'll change

Start with modules that change often and break a lot when they're wrong.

2

Recover the intent

Ask the agent what the module appears to do. Treat its answer as a draft.

3

Pin hidden contracts

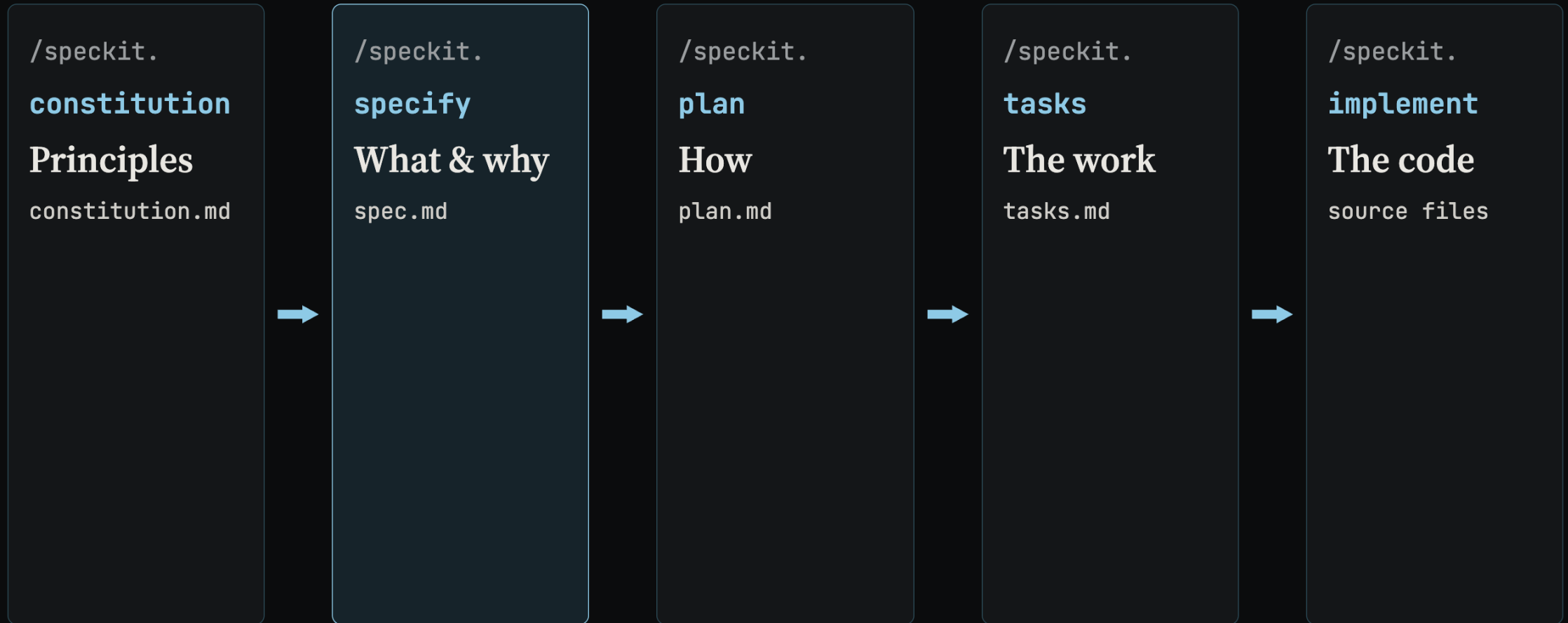
Write characterization tests and list observed behaviors first.

09 SDD frameworks

SDD frameworks at a glance

Framework	Workflow	Best for
GitHub Spec Kit	Constitution → specify → plan → tasks → implement	Open source; works with most coding agents
Kiro (AWS)	Requirements (EARS) → design → tasks	An IDE with specs built in
OpenSpec	Propose a change → apply → archive	Lightweight work on existing code
BMAD Method	Analyst, PM, architect and developer agents	Heavy, simulates a whole team
Tessl	One spec per code file; code is generated	Experiments with spec-as-source

The Spec Kit artifact chain



Lab 2: Spec Kit, about 45 minutes

1

Set up

Install specify-cli, clone repo-analysis-sprint, run specify init.

2

Constitution

Strict TypeScript, Vitest tests, lint and test before every PR.

3

Specify, plan, tasks

Turn the risk-badge story into spec.md, plan.md, tasks.md.

4

Implement and verify

Lint and tests pass. Open a PR that links your specs/ folder.

Three things to remember

01

**The substrate
changed. The
discipline didn't.**

Clear intent, testable
criteria, step-by-step
verification.

02

**Speed amplifies
everything.**

A good spec gets built fast.
So does a bad one.

03

**Match the ceremony
to the risk.**

Plan Mode for small tasks;
a living spec for features
that last.

Questions?

Next week: *Anatomy of Coding Agents*